



ARQUITETURA DO SISTEMA: PERSEUS

Alyce Mota da Silva - RA: 424107553

Everton da Fonseca Durães - RA: 1726109293

Gabriel Ferreira De Lima - RA: 924112094

João Arthur Alves de Oliveira - RA: 924107841

João Vitor Caporrino Souza - RA: 92410060

Kathlyn Faustina de Paula Souza - RA: 924104900

Layza Fernandes Silva - RA: 924104679

Roger Henrique Alves Campos - RA: 924100757

Ruan Ferreira Silva - RA: 924103220

Vinícius Soares Barbosa - RA: 924202197

Vitoria Antonio Dias - RA: 1726108233

1. INTRODUÇÃO

Este documento apresenta a arquitetura do sistema Perseus, uma aplicação voltada à mobilidade urbana, desenvolvida utilizando Node.js, Express, TypeScript, Vite, Leaflet e Supabase.

O objetivo deste documento é descrever a estrutura do sistema utilizando o modelo C4, abordando os níveis de Contexto, Containers e Componentes, além de apresentar a modelagem de dados utilizada para o armazenamento e gerenciamento das informações da aplicação.

O sistema integra funcionalidades relacionadas ao transporte público, fornecendo informações sobre linhas, paradas, rotas, geolocalização e preferências dos usuários. A plataforma foi concebida para auxiliar na tomada de decisões durante os deslocamentos urbanos, promovendo maior previsibilidade, eficiência e praticidade.

2. OBJETIVO DO SISTEMA

O sistema Perseus tem como objetivo melhorar a experiência dos usuários no transporte urbano por meio da integração de informações relevantes para o planejamento e acompanhamento de deslocamentos.

A aplicação disponibiliza funcionalidades relacionadas à consulta de linhas e paradas do transporte público, armazenamento de locais favoritos, gerenciamento de perfil de usuário e acesso a informações geográficas que auxiliam na tomada de decisões durante o trajeto.

O sistema busca oferecer maior previsibilidade, eficiência e conveniência aos usuários, contribuindo para uma experiência de mobilidade urbana mais organizada, acessível e personalizada.

3. VISÃO GERAL DA ARQUITETURA

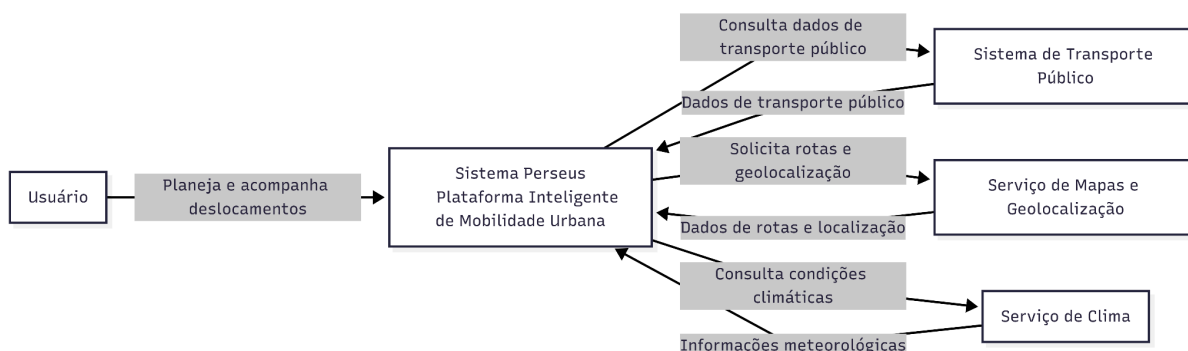
A arquitetura do sistema foi definida com base no modelo C4, permitindo uma visualização progressiva da aplicação, desde uma visão geral do sistema até seus componentes internos.

O sistema é composto por uma interface web desenvolvida em HTML, CSS e TypeScript, utilizando Vite para gerenciamento do ambiente de desenvolvimento e Leaflet para funcionalidades de mapas e geolocalização. O backend foi implementado em Node.js utilizando TypeScript e o framework Express, sendo responsável pelo processamento das regras de negócio, integração com serviços externos e comunicação com a camada de persistência de dados.

Para o armazenamento persistente das informações é utilizada a plataforma Supabase, responsável pelo gerenciamento dos dados relacionados aos usuários, perfis, locais salvos, pesquisas, eventos e demais informações necessárias para o funcionamento da aplicação.

A arquitetura foi projetada para suportar funcionalidades relacionadas à mobilidade urbana, incluindo consulta de informações sobre transporte público, gerenciamento de locais favoritos, geolocalização, personalização da experiência do usuário e integração com serviços externos, promovendo uma experiência mais eficiente e organizada durante os deslocamentos urbanos.

4. DIAGRAMA C4 — NÍVEL CONTEXTO (C1)



Descrição dos elementos:

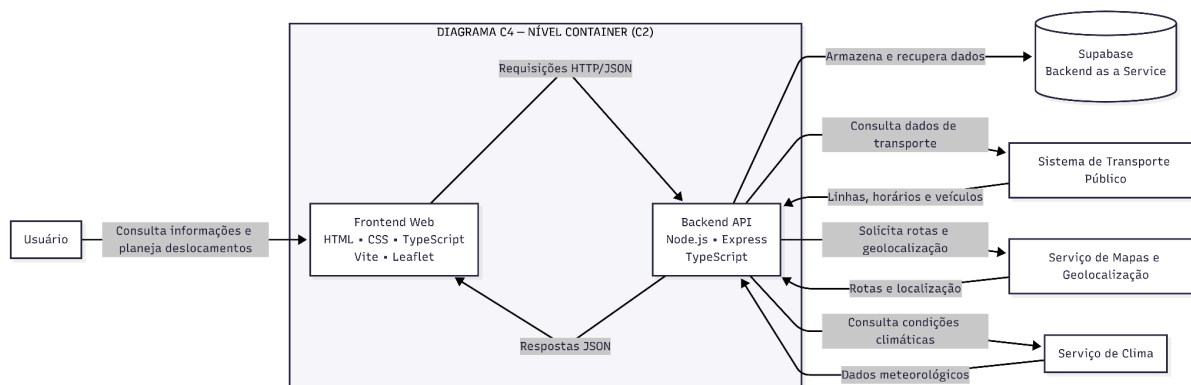
- **Usuário:** pessoa que utiliza o sistema para consultar informações de transporte público, acompanhar informações de mobilidade urbana e acessar recursos que auxiliam seus deslocamentos.
- **Sistema Perseus:** aplicação responsável por integrar informações relacionadas à mobilidade urbana, disponibilizando dados sobre transporte público, geolocalização, locais de interesse e demais informações relevantes para o planejamento dos deslocamentos urbanos.
- **Sistema de Transporte Público:** serviço externo responsável por fornecer informações sobre linhas, veículos e dados operacionais do transporte público.
- **Serviço de Mapas e Geolocalização:** serviço externo utilizado para cálculo de rotas, localização e apoio à navegação dos usuários.
- **Serviço de Clima:** serviço externo previsto na arquitetura da aplicação para fornecer informações meteorológicas que possam auxiliar os usuários durante seus deslocamentos urbanos.

O diagrama de contexto apresenta uma visão geral do Sistema Perseus e suas interações com usuários e sistemas externos.

O usuário utiliza a plataforma para planejar e acompanhar seus deslocamentos urbanos, consultando informações relevantes para sua mobilidade.

Para fornecer essas informações, o sistema integra dados provenientes de serviços de transporte público e geolocalização, além de possibilitar a integração com serviços externos complementares, contribuindo para uma experiência mais eficiente, previsível e informada durante os deslocamentos urbanos.

5. DIAGRAMA C4 — NÍVEL CONTAINER (C2)



Descrição dos Containers

- **Frontend Web:** interface acessada pelos usuários para consulta de informações sobre mobilidade urbana, transporte público, locais salvos e acompanhamento de informações relacionadas aos deslocamentos. Desenvolvida utilizando HTML, CSS, TypeScript, Vite e Leaflet para exibição de mapas e informações geográficas.
- **Backend API:** responsável pelo processamento das regras de negócio da aplicação, integração com serviços externos e comunicação com o banco de dados. Desenvolvido em Node.js utilizando TypeScript e o framework Express.
- **Banco de Dados Supabase:** plataforma responsável pelo armazenamento persistente e gerenciamento das informações utilizadas pelo sistema, incluindo dados de usuários, perfis, configurações de usuário, locais salvos, pesquisas, paradas, eventos e demais informações necessárias para o funcionamento da aplicação.
- **Serviço de Mapas e Geolocalização:** fornece informações de localização e recursos geográficos utilizados pelo sistema para apoiar a visualização e consulta de informações relacionadas à mobilidade urbana.
- **Sistema de Transporte Público:** disponibiliza informações relacionadas às linhas, paradas e operação do transporte público utilizadas pela aplicação.
- **Serviço de Clima:** serviço externo previsto na arquitetura da aplicação para fornecer informações meteorológicas que possam auxiliar os usuários no planejamento de seus deslocamentos urbanos.

O diagrama de containers apresenta a divisão interna da arquitetura do Sistema Perseus em seus principais módulos de execução.

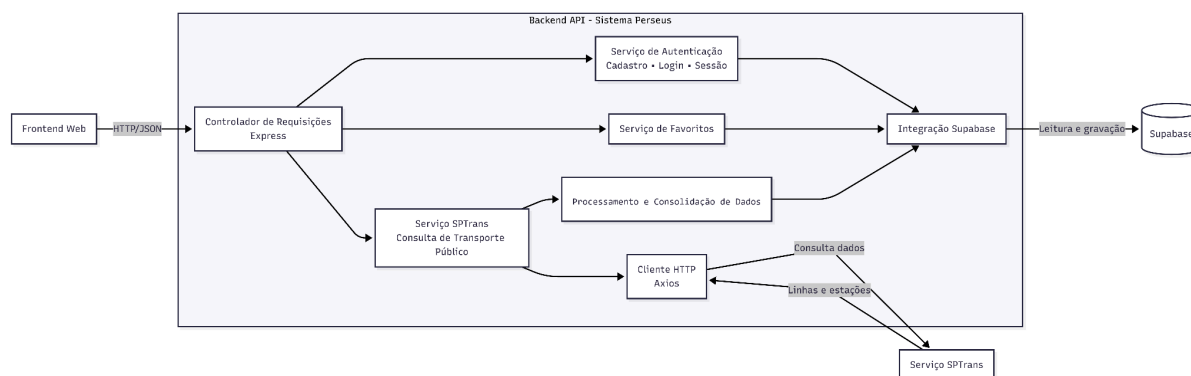
Os usuários interagem com o sistema por meio do Frontend Web, responsável pela apresentação das informações e coleta das solicitações realizadas na plataforma. As requisições são encaminhadas para a Backend API, que executa as regras de

negócio, processa os dados recebidos e realiza a integração com os serviços externos.

A Backend API consulta informações junto aos serviços de transporte público e geolocalização, processando e consolidando os dados necessários para a aplicação. A arquitetura também prevê a integração com serviços externos complementares, como informações meteorológicas, para apoiar futuras funcionalidades relacionadas à mobilidade urbana.

Essa organização promove uma arquitetura modular, facilitando a manutenção, evolução e integração de novos serviços ao sistema.

6. DIAGRAMA C4 — NÍVEL COMPONENTES (C3)



Componentes do Backend API

- **Controlador de Requisições (Express):** responsável por receber e processar as requisições HTTP enviadas pelo Frontend Web, encaminhando-as para os componentes adequados da aplicação.
- **Serviço de Autenticação:** responsável pelo cadastro, login e gerenciamento das sessões dos usuários, garantindo o controle de acesso às funcionalidades da plataforma.
- **Serviço SPTrans:** componente responsável pela integração com a API da SPTrans, realizando consultas e obtendo informações relacionadas ao transporte público, como linhas, paradas e demais dados operacionais.
- **Serviço de Favoritos:** responsável pelo gerenciamento dos locais salvos e preferências dos usuários, permitindo acesso rápido às informações frequentemente utilizadas na plataforma.
- **Processamento de Dados:** realiza o tratamento, organização e consolidação dos dados obtidos dos serviços externos e da plataforma Supabase, preparando as informações para utilização pela aplicação.
- **Integração Supabase:** responsável pela comunicação com a plataforma Supabase, realizando operações de armazenamento, consulta e recuperação das informações persistentes do sistema.
- **Cliente HTTP (Axios):** utilizado para realizar chamadas a APIs e serviços externos necessários para o funcionamento da aplicação.

O diagrama de componentes apresenta a estrutura interna do Backend API do Sistema Perseus. O controlador de requisições recebe as solicitações provenientes do Frontend Web e as encaminha para os componentes responsáveis pelo processamento das informações. O Serviço de Autenticação realiza o gerenciamento de usuários e sessões, enquanto o Serviço SPTrans integra a aplicação aos serviços de transporte público, disponibilizando dados sobre linhas, paradas e demais informações relacionadas ao transporte público.

O Serviço de Favoritos permite o armazenamento e recuperação das preferências dos usuários, proporcionando uma experiência mais personalizada. O componente de Processamento de Dados é responsável por consolidar e organizar as informações obtidas dos serviços externos e do banco de dados. A Integração Supabase realiza a persistência das informações da aplicação, enquanto o Cliente HTTP (Axios) possibilita a comunicação com APIs externas.

Essa organização em componentes promove maior modularidade, facilita a manutenção do código, melhora a reutilização de funcionalidades e contribui para a evolução futura do sistema, permitindo a integração de novos recursos relacionados à mobilidade urbana.

7. DIAGRAMS AS CODE

Os diagramas da arquitetura foram desenvolvidos utilizando a linguagem Mermaid, permitindo a criação, visualização e manutenção dos diagramas por meio de código textual. Essa abordagem facilita o versionamento da documentação juntamente com o código-fonte do projeto, contribuindo para a rastreabilidade das alterações e para a consistência entre a arquitetura documentada e a implementação do sistema.

A utilização do conceito de Diagrams as Code possibilita que os diagramas sejam atualizados de forma simples e padronizada, favorecendo a manutenção da documentação arquitetural ao longo da evolução do Sistema Perseus.

Código Mermaid utilizado para a geração do Diagrama de Contexto (C1):

```
---
config:
  layout: dagre
---

flowchart LR
    U["Usuário"] -- Planeja e acompanha deslocamentos --> P["Sistema Perseus  
Plataforma Inteligente de Mobilidade Urbana"]
    P -- Consulta dados de transporte público --> TP["Sistema de  
Transporte Público"]
    TP -- Dados de transporte público --> P
    P -- Solicita rotas e geolocalização --> MAP["Serviço de  
Mapas e Geolocalização"]
    MAP -- Dados de rotas e localização --> P
    P -- Consulta condições climáticas --> CLIMA["Serviço de  
Clima"]
    CLIMA -- Informações meteorológicas --> P
```

Código Mermaid utilizado para a geração do Diagrama de Contexto (C2):

```
flowchart LR
    U["Usuário"]
    subgraph P["Sistema Perseus"]
        FE["Frontend Web  
HTML • CSS • TypeScript  
Vite • Leaflet"]
        API["Backend API  
Node.js • Express  
TypeScript"]
        FE -->|Requisições HTTP/JSON| API
    end
```

```

    API -->|Respostas JSON| FE
end
DB[("Supabase
Banco de Dados")]
TP["Sistema de Transporte Público"]
MAP["Serviço de Mapas e Geolocalização"]
CLI["Serviço de Clima"]
U -->|Consulta informações e planeja deslocamentos| FE
API -->|Armazena e recupera dados| DB
API -->|Consulta dados de transporte| TP
TP -->|Linhas, horários e veículos| API
API -->|Solicita rotas e geolocalização| MAP
MAP -->|Rotas e localização| API
API -->|Consulta condições climáticas| CLI
CLI -->|Dados meteorológicos| API

```

Código Mermaid utilizado para a geração do Diagrama de Contexto (C3):

```

flowchart LR
    subgraph API["Backend API - Sistema Perseus"]
        CTRL["Controlador de Requisições Express"]
        AUTH["Serviço de Autenticação Cadastro • Login • Sessão"]
        SP["Serviço SPTrans<br>Consulta de Transporte Público"]
        FAV["Serviço de Favoritos"]
        PROC@{ label: "<pre class=\"overflow-visible!\" data-start=\"1154\" data-end=\"1203\"><pre class=\"cm-content\"><code>Processamento e Consolidação de Dados</code></pre></pre><br><p data-start=\"1205\" data-end=\"1265\"></p>" }
        AX["Cliente HTTP Axios"]
        SUP["Integração Supabase"]
    end

    CTRL --> AUTH & SP & FAV
    AUTH --> SUP
    FAV --> SUP
    SP --> AX & PROC
    PROC --> SUP
    FE["Frontend Web"] -- HTTP/JSON --> CTRL
    AX -- Consulta dados --> EXT["Serviço SPTrans"]
    EXT -- Linhas e estações --> AX

```

```
SUP -- Leitura e gravação --> DB[("Supabase")]
```

```
PROC@{ shape: rect}
```

8. MODELAGEM DE DADOS — SUPABASE

O Sistema Perseus utiliza o Supabase como plataforma de persistência de dados, responsável pelo armazenamento das informações necessárias para as funcionalidades de mobilidade urbana, autenticação e personalização da experiência dos usuários.

A modelagem de dados foi estruturada em entidades relacionadas aos usuários, locais, pesquisas, eventos e informações geográficas utilizadas pela aplicação.

Principais Entidades

Perfil

A entidade Perfil armazena informações complementares dos usuários cadastrados na plataforma.

Campos:

- id: identificador único do usuário.
- nome: nome do usuário.
- foto_perfil: imagem associada ao perfil.
- cpf: documento de identificação do usuário.
- criado_em: data de criação do registro.

Configurações do Usuário

Responsável pelo armazenamento das preferências de utilização da plataforma.

Campos:

- usuario_id: referência ao usuário.
- tema: tema visual selecionado.
- mostrar_localizacao: define se a localização do usuário será exibida.
- mostrar_paradas: define se as paradas serão exibidas na interface.
- criado_em: data de criação do registro.

Paradas

Armazena informações relacionadas às paradas de transporte público utilizadas pela aplicação.

Campos:

- id: identificador único da parada.
- codigo: código da parada.
- nome: nome da parada.
- endereco: endereço da parada.
- latitude: coordenada geográfica de latitude.
- longitude: coordenada geográfica de longitude.
- criado_em: data de criação do registro.

Dados de Geolocalização

Responsável pelo armazenamento das informações geográficas utilizadas pela aplicação.

Campos:

- id: identificador único do registro.
- endereco: endereço associado à localização.
- latitude: coordenada geográfica de latitude.
- longitude: coordenada geográfica de longitude.
- criado_em: data de criação do registro.

Pesquisas

Armazena o histórico de pesquisas realizadas pelos usuários.

Campos:

- id: identificador único da pesquisa.
- termo: termo pesquisado.
- latitude: coordenada geográfica associada à pesquisa.
- longitude: coordenada geográfica associada à pesquisa.
- criado_em: data de criação do registro.

Locais Salvos

Permite o armazenamento de locais favoritos dos usuários.

Campos:

- id: identificador único do local salvo.

- usuario_id: referência ao usuário proprietário.
- nome: nome do local.
- endereco: endereço do local.
- latitude: coordenada geográfica de latitude.
- longitude: coordenada geográfica de longitude.
- criado_em: data de criação do registro.

Eventos

Armazena registros de eventos relacionados ao funcionamento da aplicação.

Campos:

- id: identificador único do evento.
- tipo: categoria do evento registrado.
- detalhes: informações adicionais do evento.
- criado_em: data de criação do registro.

A modelagem de dados foi desenvolvida para oferecer suporte às funcionalidades de autenticação, gerenciamento de perfil, armazenamento de locais favoritos, consulta de paradas de transporte público, geolocalização e histórico de pesquisas, garantindo persistência, integridade e recuperação eficiente das informações utilizadas pelo Sistema Perseus.

CONCLUSÃO

O desenvolvimento da arquitetura utilizando o modelo C4 permitiu uma visão clara e organizada do Sistema Perseus em diferentes níveis de abstração, facilitando a compreensão de seus componentes, responsabilidades, integrações e fluxos de comunicação.

A utilização de tecnologias como Node.js, Express, TypeScript, Vite, Leaflet e Supabase possibilitou a construção de uma aplicação voltada à mobilidade urbana, integrando informações relacionadas ao transporte público, geolocalização e gerenciamento de dados dos usuários.

A arquitetura proposta contribui para a modularidade, organização e manutenção da aplicação, promovendo uma separação adequada das responsabilidades entre a interface do usuário, os serviços de negócio, as integrações externas e a camada de persistência de dados.

Além disso, a documentação arquitetural desenvolvida por meio do modelo C4 e da abordagem Diagrams as Code facilita a compreensão da estrutura do sistema, auxiliando futuras evoluções e integrações de funcionalidades voltadas à melhoria da experiência dos usuários durante seus deslocamentos urbanos.